

CSE_3523

8086 Microprocessor Architecture

Microprocessor, microcontroller and embedded system

Course code: CSE-3523

Credit Hour: 3

Md. Mujibur Rahman Maruf

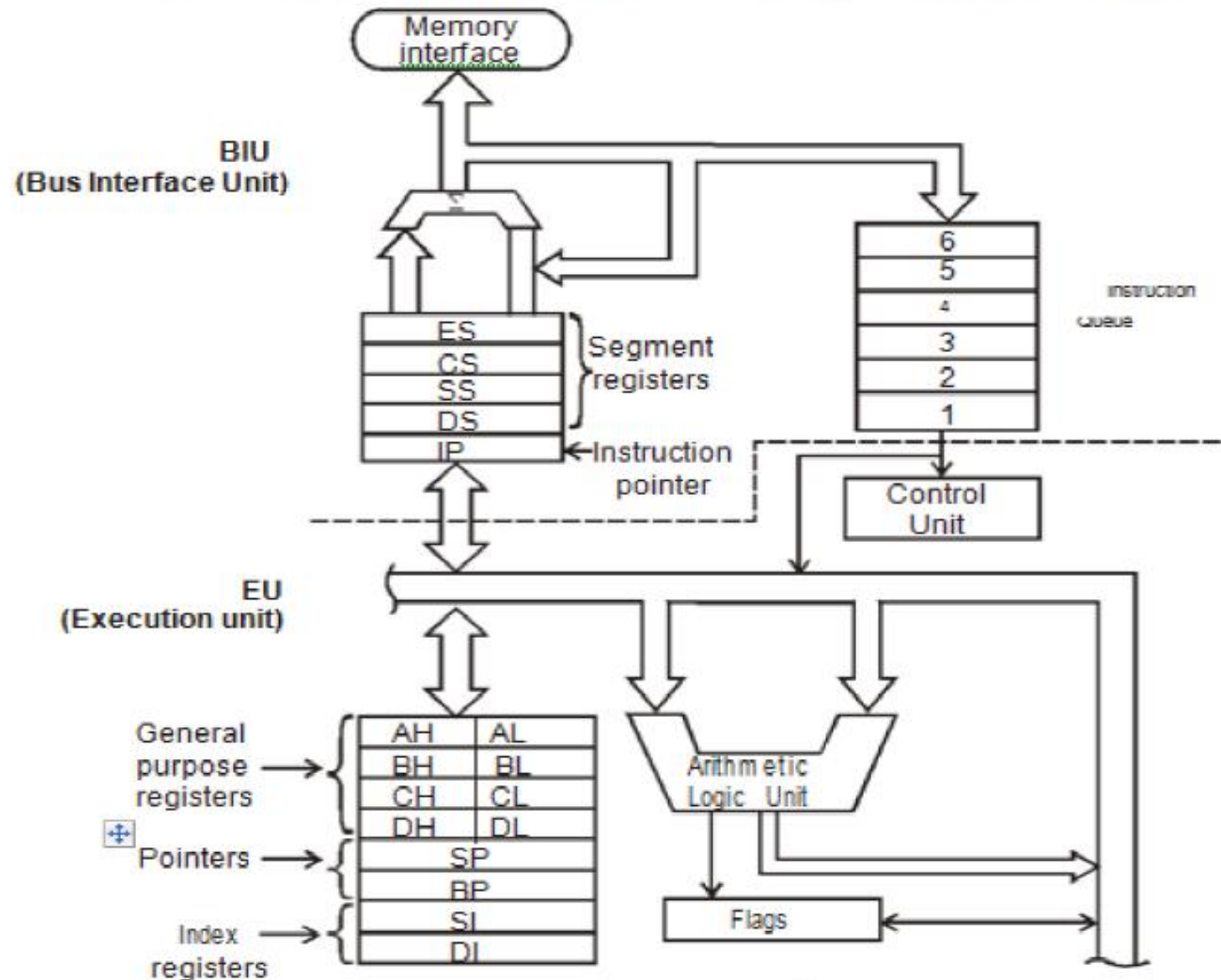
Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

8086 Microprocessor Architecture



8086 Microprocessor Architecture

BIU = “bring/send things”, EU = “do the work”.

the 8086 processor are partitioned logically into two processing units

For the 8086 specifically, Intel organizes those internal CPU functions into two big sections:

- **Bus Interface Unit (BIU)**

The BIU fetches instructions, reads data from memory and ports, and writes data to memory and I/O ports.

- **Execution Unit (EU)**

EU receives program instruction codes and data from the BIU, executes these instructions and stores the results either in the general registers or output them through the BIU.

8086 Microprocessor Architecture



The BIU contains

- Segment registers
- Instruction pointer
- Instruction queue

The EU contains

- ALU
- General purpose registers
- Index registers
- Pointers
- Flag register

Register

- ❑ A register is **one of a small set of data holding places** that are part of a computer processor.
- ❑ Registers are two types:
 1. General-purpose or multipurpose Registers
 - These registers hold various data sizes and used for almost any purpose.
 2. Special purpose Registers
 - Used in special purposes.

Segment Register

- ❑ Segment Registers are **additional registers**, **generate memory addresses** when combined with other registers in the microprocessor.

❑ In a word, *segment registers are section of memory.*

CS	CODE Segment
DS	DATA Segment
ES	EXTRA Segment
SS	STACK Segment

Each segment register stores the starting address of a memory segment, and that segment can be at most 64 KB.

Segment Register

1. Code segment (CS)

- The code segment is a section of memory that **holds the code** (programs and procedures) used by the microprocessor.
- The code segment (CS) register defines the starting address of the section of memory holding **code**.
- The code segment size is limited to max 64KB.

CS:IP = 1000:0000

Physical address = $1000H \times 10H + 0000H = 10000H$

Segment Register

2. Data segment (DS)

- The data segment **contains most data** used by a program.
- The data segment (DS) register defines the starting address of the section of memory holding **data**.
- The data segment size is limited to max 64KB.
- Data are accessed in the data segment by an offset address or the contents of other registers that hold the offset address.

Segment Register

3. Extra segment (ES)

- ES is an **additional data segment** used by some of the string instructions to hold destination data.
- The extra segment (ES) register defines the starting address of the section of memory used as **extra segment**.
- Can be used as an extra/additional segment for code or data.
- Max size of the segment is limited to 64KB.

Segment Register

4. Stack segment (SS)

- The stack segment defines the **area of memory used for the stack**.
- The stack segment (SS) register defines the starting address of the section of memory used as **stack**.
- The stack entry point is determined by the stack segment and stack pointer registers.

General-purpose or multipurpose Registers

Bit position

15

8 7

0

- ❑ The MP has 07 general purpose registers.
- ❑ Each registers are 16-bit in size, while some registers can be divided into two 8-bit register.
- ❑ The acceptable 8-bit register pairs are AH-AL, BH-BL, CH-CL, DH-DL. These pairs forms AX, BX, CX, DX registers, respectively.

AX	AH	AL	Accumulator
BX	BH	BL	Base index
CX	CH	CL	Count
DX	DH	DL	Data
	BP		Base pointer
	SI		Source index
	DI		Destination index

If SI tells:

"Where am I reading from?"

DI tells:

"Where am I writing to?"

General-purpose or multipurpose Registers

1. AX register

- Called 16-bit accumulator, while AH/AL is 8-bit accumulator.
- Used for instructions such as **Multiplication & division**.

2. BX register

- 16-bit Base index register, while BH/BL is 8-bits.
- Sometimes **holds offset address**.

`MOV BX, 2000H`
`MOV AX, [BX]`

Offset is the displacement of the memory location from the starting location of the segment.
Go to memory at offset 2000H and take the value from there.

3. CX register

- 16-bit Count register, while CH/CL is 8-bits.
- **Holds count for various instructions** such as SHIFT, ROTATE, LOOP, etc.

`MOV CX, 5`

Imagine a loop that needs to run 5 times.

CX can store that count.

After each loop, the count decreases until it reaches zero.

General-purpose or multipurpose Registers

4. DX register

- 16-bit Data register, while DH/DI is 8-bits.
- **Holds a part of the result** from multiplication or part of the dividend before a division. DX helps when AX needs extra room.
- Need for I/O devices.

5. BP register

- 16-bit Base Pointer register. MOV AX, [BP]
Read the value from the stack location BP is pointing to.
- **Points to a memory location.**

6. SI register

- 16-bit Source Index register. H E L L O
↑
SI Then SI moves forward as each byte/word is processed.
- **Addresses source string** data for the string instructions.

Special-purpose Registers

❑ Special-purpose registers include IP, SP, FLAGS.

1. IP register

- Called 16-bit Instruction Pointer.
- The Instruction pointer, which **points to the next instruction in a program**, is used by the microprocessor to find the next sequential instruction in a program located within the code segment.

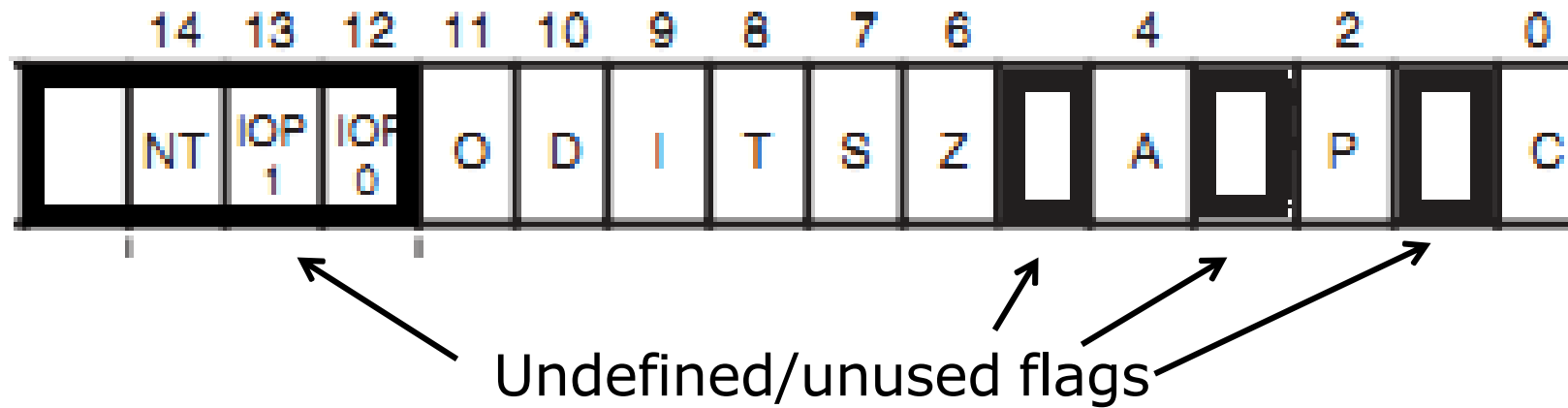
2. SP register

- 16-bit Stack Pointer, **addresses an area of memory called the stack**.
- The stack memory stores data through this pointer.

FLAGS register

- A flag register (F or FL) is a 16-bit register, which indicates some condition produced by the execution of an instruction and controls certain operation of the MP.
- Flag register contains 09 active flags. Among them 06 are status flags (indicate some condition produced by an instruction) and 03 are control flags (control certain operation of processor). These are -
 - ✓ Carry (C)
 - ✓ Parity (P)
 - ✓ Auxiliary carry (A)
 - ✓ Zero (Z)
 - ✓ Sign (S)
 - ✓ Trap (T)
 - ✓ Interrupt (I)
 - ✓ Direction (D)
 - ✓ Overflow (O)

FLAGS register



CPAZS → 0,2,4,6,7

TIDO → 8,9,10,11

0011 1010 1101 0101 .

FLAGS register

1. C (Carry) Flag

- It holds the carry after addition or the borrow after subtraction.

FFH
+ **FFH**
—
FEH

1111 1111
1111 1111
—
1 1111 1110



Here, **C=1**
If No carry, **C=0**

FLAGS register

2. P (Parity) Flag

- Parity is the count of ones in register.
 - Expressed as even or odd.
 - Logic 0 for odd parity and logic 1 for even parity.
 - If a number contains no one bits, it has even parity.

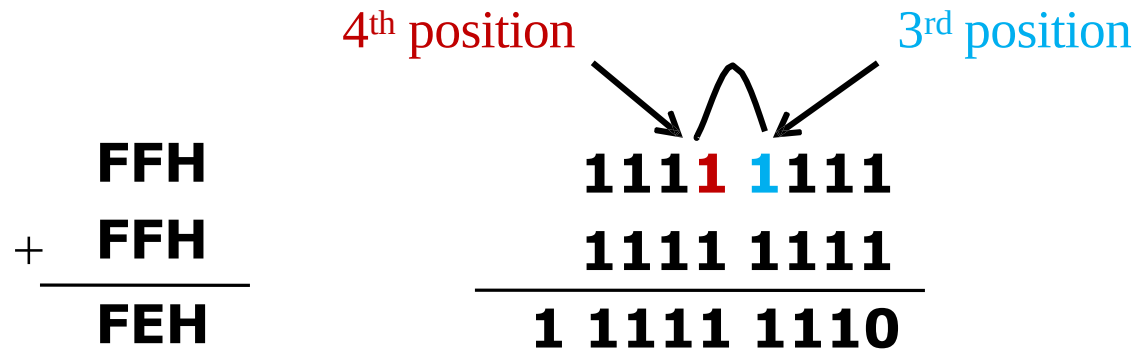
$$\begin{array}{r} \text{FFH} \\ + \text{FFH} \\ \hline \text{FEH} \end{array}$$
$$\begin{array}{r} 1111 \ 1111 \\ 1111 \ 1111 \\ \hline 1 \ 1111 \ 1110 \end{array}$$

Here, No. of "1"=7,
So, Odd parity i.e., **P=0**
If even ~~carry~~ parity, P=1

FLAGS register

3. A (Auxiliary carry) Flag

- The auxiliary carry **holds the carry (half-carry)** after addition or the borrow after subtraction **between bits positions 3 and 4**.



Here, "1" transfer between 3rd and 4th positions bits. So, **A=1** otherwise, **A=0**

4. Zero Flag

FLAGS register

5. S (Sign) Flag

- The sign flag **holds the arithmetic sign** after an arithmetic or a logical operation.
- If $S = 1$, the sign bit is set and the result is **negative (-ve)**.
- If $S = 0$, the sign bit is not set and the result is **positive (+ve)**.
- Usually sign bit is the leftmost bit.

6. T (Trap/Trace) Flag

- It enables trapping through an on-chip debugging (checking error) facility.
- $T = 1$, Debugging enable
- $T = 0$, Debugging disable

Should the CPU execute normally, or stop after every instruction so a debugger can inspect what happened?

$T = 1$

the CPU enters single-step / debugging behavior:

Execute Instruction 1

↓
STOP → debugger can inspect things

Execute Instruction 2

↓
STOP → debugger can inspect things

$T = 0$

The CPU executes continuously:

Instruction 1
Instruction 2
Instruction 3
Instruction 4

FLAGS register

7. I (Interrupt) Flag

I = 1 → INTR enabled → CPU can accept interrupt requests
I = 0 → INTR disabled → CPU ignores/masks those requests

Okay, I'll temporarily handle your request, then come back to what I was doing.

- The interrupt flag controls the operations of the INTR (Interrupt request) input pin. If I = 1, the INTR pin is enabled; INTR pin is disabled with I = 0.

8. D (Direction) Flag

- The D flag selects either the increment or decrement mode for DI and/or SI registers during string instructions.
 - If D = 1, the registers are automatically decremented; Apple
 - If D = 0, the registers are automatically incremented. apple backwards elppa
- **Decrement mode indicates Set direction and write with STD** (set direction) instruction. **Increment mode indicates Clear direction and write with CLD** (clear direction) instruction. D=1 D=0

FLAGS register

9. O (Overflow) Flag

- An overflow indicates that the result has exceeded the capacity of the machine.
- Overflow occurs when signed numbers are added or subtracted.
- Mainly occurs during multiplication operation.
 - If $O = 1$, Overflow occurs;
 - If $O = 0$, No overflow.

The flag register value after execution of an instruction is

3AD5H. Determine the status of the following flags:

Carry Flag (CY), Parity Flag (P), Auxiliary Carry Flag (AC), Zero
Flag (Z), Sign Flag (S) 0011 1010 1101 0101

Memory-addressing techniques

- Memory Address is the location of a program counter in a microprocessor.
- When a microprocessor is booted up, it loads the microcode that contains the addresses of the processor's registers.
- These registers are used to hold values that the microprocessor can read and write.
- The microprocessor places a value at the beginning of each register in the stack and places a value at the end of each register in the stack.
- When the microprocessor is called, it fetches the value at the location in the microcode that corresponds to the register on the stack.

Real Mode Memory Addressing

- ❑ Real mode operation allows MP to address only the first 1MB of memory space.
 - First 1MB of memory is called either real memory or conventional memory system.
 - DOS operating system requires MP to operate in real mode.
 - Windows does not use real mode.

Segment and Offset address

- ❑ A combination of a segment address and an offset address accesses a memory location in real mode. All real mode **memory addresses must consist of segment add. plus offset address.**
 - The segment address, located within one of the segment registers, defines the **beginning address** of any 64K-byte memory segment.
 - The offset address is also held in a register and **selects any location** within the 64K byte memory segment. An offset address is also sometimes referred to as displacement or logical address.
- ❑ A memory location pointed by a segment address of 1000H and offset address of 2000H is written as 1000:2000.
 - What is the actual/physical memory location for 1000:2000?

Segment and Offset address

❑ Calculation of physical address from segment : offset pair 1000:2000

1. Append **0H** to the right of segment value.

1 0 0 0 **0**

2. Add offset value with the appended segment value.

2 0 0 0

3. The result is the physical address for the given segment : offset pair

1 2 0 0 0

$$\text{Physical address} = \text{Segment add.} \times 10 + \text{Offset}$$

If Physical address is 22004H, what will be the Segment and Offset add.?

Segment = 2200H
Offset = 0004H

Segment and Offset address

- ❑ Because a real mode segment of memory is 64K in length, once the beginning address is known, the ending address is found by adding FFFFH.
- ❑ If segment register contains 3000H, first address of segment is 30000H and last address is 30000+FFFF or 3FFFFH.

Default Segment and Offset Registers

❑ 8086 has a set of rules that apply to segments whenever memory is addressed. These rules define the segment and offset register combination.

- CS always used with IP to address next instruction in program. CS register defines start of CS and IP locates next instruction within CS.
- CS=1400H & IP=1200H, MP fetches its next instruction from memory location 15200H.

Segment	Offset	Combination	Purpose
CS	IP	CS:IP	Code / Instruction address
DS	BX,SI,DI	DS:BX/SI	Data address
ES	DI	ES:DI	String destination address
SS	SP or BP	SS:SP	Stack address

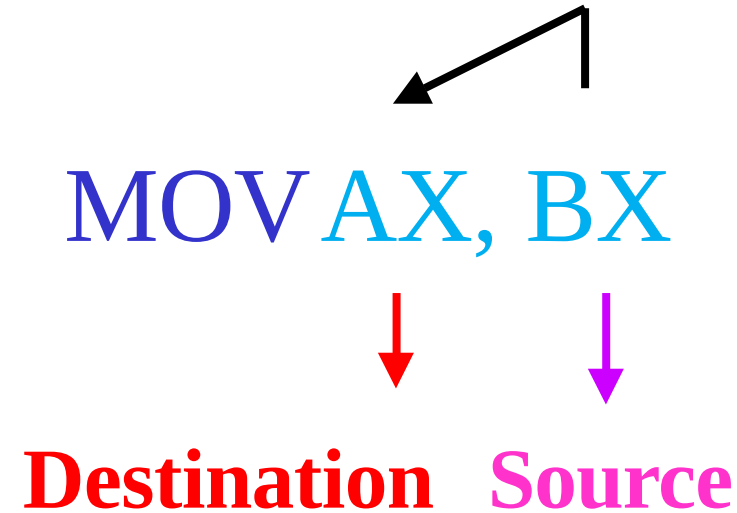
CSE_3523

Addressing Modes of 8086

Addressing modes

- ❑ The different ways in which a processor can access data are referred to as **addressing mode**.
- ❑ Addressing modes for 8086 MP are three types –
 1. Data addressing modes How do I get the DATA/operand I want?
 2. Program memory addressing modes Where is the next instruction I want to execute?
 3. Stack memory addressing modes Where in the stack do I access?

Addressing modes



MOV **AX**, **BX** instruction translates the word contents of the **source** register (**BX**) into the **destination** register (**AX**).

Types of addressing mode in 8086

subtype of data addressing mode.

1. Immediate addressing mode
2. Direct addressing mode
3. Register addressing mode
4. Register Indirect addressing mode
5. Indexed addressing mode
6. Register relative addressing mode
7. Base plus index addressing mode
8. Base relative plus index addressing mode

1: Immediate addressing mode

In this type of mode, immediate data is part of instruction and appears in the form of successive byte or bytes

❖ **Not allowed**

➤ **MOV AH, 1000H**

- instruction is not allowed. AH is only 8-bit, but 1000H needs 16 bits

MOV 75H, AL

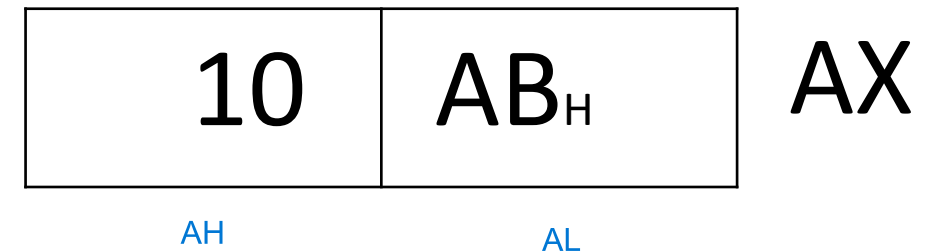
MOV [75H], AL(allowed)

- instruction is not allowed.

➤ **MOV CS, 1000H**

- Segment registers does not allow immediate data.

MOV AX, 10AB_H



2:Direct addressing mode

The memory address is written directly in the instruction.

3. Direct addressing

- ❑ Moves a byte or word between a memory location & a register.
- ❑ e.g., **MOV BX, LIST** instruction copies word-sized contents of memory location LIST into register CX.
- ❑ **MOV AX, [2004]** instruction copies word-sized contents of memory location 2004H into register AX.

Direct addressing

MOV BX, LIST

If,
LIST=2001H

BX=1312H

MOV DX, [2004]

DX=1615H

MOV [2003], DH

FF	2000
12	2001
13	2002
14 16	2003
15	2004
16	2005

❖ Find out BX, DX and show change in memory.

2:Direct addressing mode

MOV AX,[5000H]

33	22
----	----

AX

Memory

22	5000
33	5001
	5002

❖ ***Not allowed***

➤ **MOV [AX], [BX]** • memory to memory transfer is not possible.

3: Register addressing mode

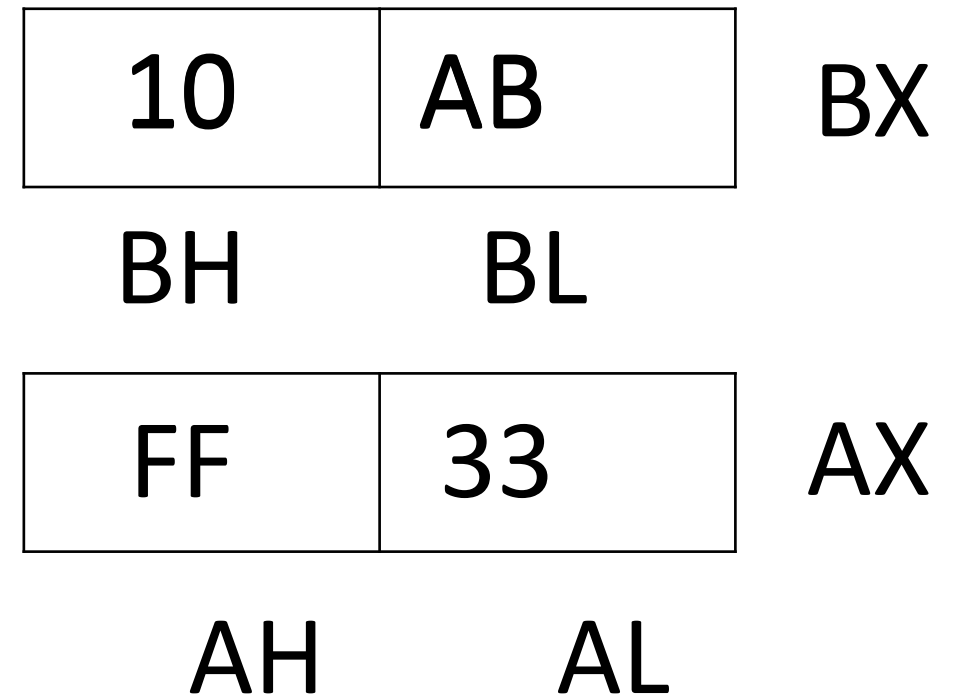
In this type of addressing mode, the data is stored in the register and it can be a 8-bit or 16-bit register. All the registers, except IP, may be used in this mode.

❖ **Not allowed**

- **MOV AX, AL** • These registers are of different sizes.
- **MOV CS, AX** • CS can not be the destination register.
- **MOV CS, DS** • Segment to segment transfer is not possible

MOV AL,BLH

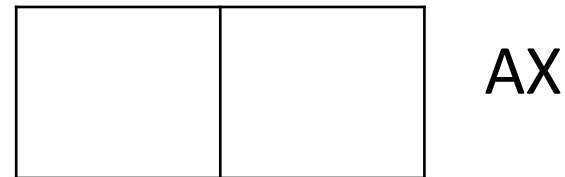
MOV AX,BXH



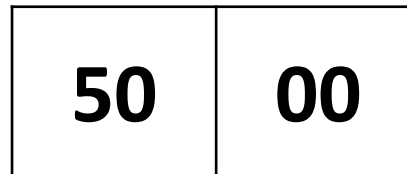
4: Register Indirect addressing mode

The address of the memory location which contains data or operand is determined in an indirect way, using the offset register.

MOV AX,[BX]



AX



BX

Memory

22	5000
33	5001
	5002

4: Register Indirect addressing mode

MOV BX, 2002H

MOV AX, 2233H

MOV [BX], AX

FF	2000
12	2001
13 33	2002
14 22	2003
15	2004
16	2005

❖ ***Not allowed***

➤ MOV AX, CS

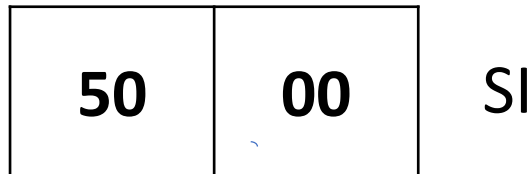
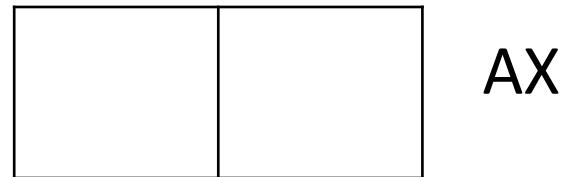
➤ MOV AX, [CS]

•CS memory itself.

5:Indexed addressing mode

In this addressing mode, offset of the operand is stored in one of the index registers. DS is the default segment for index register SI and DI.

MOV AX,[SI]



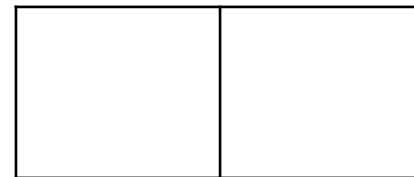
Memory

22	5000
33	5001
	5002

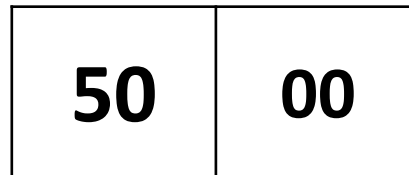
6: Register relative addressing mode

In this mode, the data is available at an effective address formed by adding an 8-bit or 16-bit **displacement** with the content of any one of the registers BX, BP, SI and DI in the default (either DS or ES) segment.

MOV AX,[BX+50H]



AX



BX

+ 50H = 5050H

Offset

Final
Index
Address

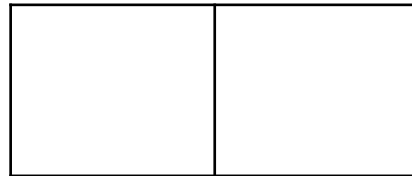
Memory

44	5050
33	5051
	5052

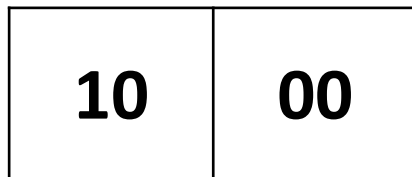
7: Base plus index addressing mode

In this mode the effective address is formed by adding content of a **base register** (any one of BX or BP) to the content of an **index register** (SI or DI). Default segment register DS. □ **MOV AX, [BX+SI]** is same as **MOV AX, [BX][SI]**.

MOV AX, [BX][SI]

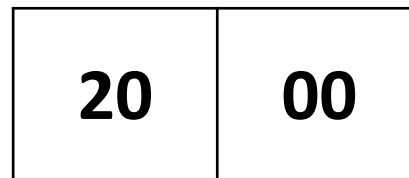


AX



BX

+



SI

= 3000H

Final
Index
Address

12	3000
34	3001
	3002

7: Base plus index addressing mode

MOV AX, [BX+SI]

If, BX=2003H

SI=0001H

AX=1615H

FF	2000
12	2001
13	2002
14	2003
15	2004
16	2005

8:Base relative plus index addressing mode

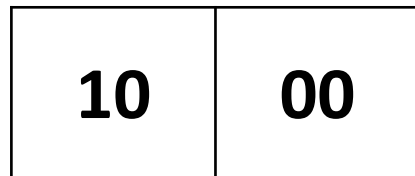
In the effective address is formed by adding an 8 or 16-bit displacement with sum of contents of any one of the **base registers** (BX or BP) and any one of the **index registers**, in a default segment.

MOV AX,50H[BX][SI]

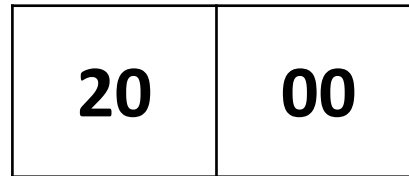


AX

50H +



BX



SI

= 3050H

Final
Index
Address

12	3050
34	3051
	3052

Reference

Chapter 2 and 3

Barry B. Brey, **The Intel Microprocessors**